NBA Expected Field-Goal Percentage — a SEMMA Project

Author: `<your name>` · **Course:** CMPE 255 · **Repo:** MeronDT/CMPE-255-Data-Mining-process · **App:** `<deployed URL>`

Data: NBA shot-chart detail (stats.nba.com), compiled by shufinskiy/nba_data (Apache-2.0; also on Kaggle). Every regular-season field-goal attempt with court coordinates, shot type, game clock and outcome. **Population:** five seasons, 2019-20 → 2023-24 (1,031,740 shots). **Out-of-time check:** 2024-25 (219,527 shots).

SEMMA (SAS Institute) organises the *technical* core of data mining into five steps. Each step below maps to the **SAS Enterprise Miner nodes** that implement it, so the same flow can be rebuilt in SAS (see the appendix).

Step	Purpose	Enterprise Miner nodes used here
Sample	draw a representative, manageable sample; partition it	Input Data · Sample · Data Partition
Explore	understand distributions, relationships, anomalies	StatExplore · MultiPlot / Graph Explore · Variable Selection · Cluster
Modify	create, select, transform variables	Drop · Transform Variables · Replacement · Interactive Binning · Impute
Model	fit competing predictive models	Regression · Decision Tree · Neural Network · Gradient Boosting · Ensemble
Assess	compare models, check reliability, score new data	Model Comparison · Cutoff · Score · Segment Profile

Headline result: a gradient-boosting model that estimates the make probability of any shot from where and how it is taken. It beats the zone-average baseline on every partition and on the unseen 2024-25 season, and is calibrated to within 0.9 percentage points in every decile. Comparing players' actual FG% with the xFG% of their shots separates shot *making* from shot *selection*, and that skill is repeatable (r = 0.68 across seasons).

How to run: *Runtime → Run all* (≈ 3–5 min on free Colab). Every figure, table, the model and the app are written to `artifacts/`.

Project context (pre-SEMMA)

SEMMA assumes the problem is already defined — unlike CRISP-DM it has no Business Understanding phase — so the problem is stated briefly here, to justify later choices.

- **Question:** given *where* and *how* a shot is taken (location, shot type, game clock), what is the probability it goes in? This is **expected field-goal percentage (xFG%)**

— a measure of **shot quality**, independent of who shoots.

- **Why it matters:** raw FG% mixes two things: *which* shots a player takes and *how well* he makes them. A center who only dunks has a high FG% without being a great shooter. Comparing actual FG% with the xFG% of the same shots separates **shot selection** from **shot making** — the same idea as moving from box-score stats to advanced metrics when judging MVP candidates.
- **Target:** `SHOT_MADE_FLAG` (1 = made). **Unit:** one field-goal attempt.
- **Success criteria:** the champion model must (1) beat a **zone-average baseline** (league FG% for the shot's official court zone) in validation **log loss**, (2) be **calibrated** (predicted ≈ actual within each probability band), and (3) hold its performance on the **unseen 2024-25 season**.

⚙️ Setup & configuration

In [1]:
```
import json, platform, sys, tarfile, time, urllib.request, warnings
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 40)
plt.rcParams["figure.figsize"] = (8, 4.5)
print(f"python {platform.python_version()} | numpy {np.__version__} | pandas {pd
```

```
python 3.13.18 | numpy 2.5.3 | pandas 3.0.5 | scikit-learn 1.9.1
```

In [2]:
```
# ============================== CONFIGURATION ==============================
URL_TEMPLATE = "https://github.com/shufinskiy/nba_data/raw/main/datasets/shotdet
POPULATION_SEASONS = [2019, 2020, 2021, 2022, 2023]   # file year = season start
OUT_OF_TIME_SEASON = 2024                              # 2024-25, scored only in
TARGET       = "SHOT_MADE_FLAG"
SAMPLE_SIZE  = 200_000          # shots drawn from the population (justified by
PARTITION    = (0.6, 0.2, 0.2)  # train / validate / test, split by GAME so shot
RANDOM_STATE = 42
# ===========================================================================

ART = Path("artifacts"); DATA, FIG, REP, MOD, APP = (ART / p for p in ("data", "
for p in (DATA, FIG, REP, MOD, APP):
    p.mkdir(parents=True, exist_ok=True)

def savefig(name):
    plt.tight_layout(); plt.savefig(FIG / f"{name}.png", dpi=120, bbox_inches="t

def draw_court(ax, color="#555", lw=1.2):
    """NBA half court in shot-chart units (tenths of feet, hoop at 0,0)."""
    from matplotlib.patches import Circle, Rectangle, Arc
    for p in [Circle((0, 0), 7.5, fill=False), Rectangle((-30, -7.5), 60, 0), Re
              Arc((0, 142.5), 120, 120, theta1=0, theta2=180), Arc((0, 0), 80, 8
              Rectangle((-220, -47.5), 0, 140), Rectangle((220, -47.5), 0, 140),
              Rectangle((-250, -47.5), 500, 470, fill=False)]:
```

```
        p.set_edgecolor(color); p.set_linewidth(lw); ax.add_patch(p)
    ax.set_xlim(-250, 250); ax.set_ylim(-47.5, 422.5); ax.set_aspect("equal"); a
```

S — Sample

Goal: extract a portion of the data that is large enough to contain the significant information, yet small enough to process quickly — and partition it for honest model assessment.

S.1 Input Data

In [3]:
```
def load_season(season):
    path = DATA / f"shotdetail_{season}.csv"
    if not path.exists():                                    # download once, then
        archive = DATA / f"shotdetail_{season}.tar.xz"
        urllib.request.urlretrieve(URL_TEMPLATE.format(season=season), archive)
        with tarfile.open(archive, "r:xz") as t:
            t.extractall(DATA)
        archive.unlink()
    d = pd.read_csv(path, dtype={c: "category" for c in ["GRID_TYPE", "PLAYER_NA
                                                         "SHOT_TYPE", "SHOT_ZONE
    d.insert(0, "SEASON", f"{season}-{str(season + 1)[-2:]}")
    return d

t0 = time.time()
population = pd.concat([load_season(s) for s in POPULATION_SEASONS], ignore_inde
for c in population.select_dtypes("object").columns:
    population[c] = population[c].astype("category")
print(f"Population: {len(population):,} shots x {population.shape[1]} columns, {
      f"({time.time() - t0:.0f}s, {population.memory_usage(deep=True).sum() / 1e
display(population.groupby("SEASON", observed=True).agg(shots=(TARGET, "size"),
population.head(3)
```

```
Population: 1,031,740 shots x 25 columns, 5,829 games (5s, 121 MB)
```

SEASON	shots	games	fg_pct
2019-20	188116	1059	0.460
2020-21	190983	1080	0.466
2021-22	216722	1230	0.461
2022-23	217218	1230	0.475
2023-24	218701	1230	0.474

Out[3]:

	SEASON	GRID_TYPE	GAME_ID	GAME_EVENT_ID	PLAYER_ID	PLAYER_NAME	TEA
0	2019-20	Shot Chart Detail	21900014	9	1629631	De'Andre Hunter	16106
1	2019-20	Shot Chart Detail	21900014	11	203458	Alex Len	16106
2	2019-20	Shot Chart Detail	21900014	14	1629027	Trae Young	16106

The 2019-20 (COVID suspension and bubble) and 2020-21 (72-game) seasons have fewer games, but shooting barely differs between seasons (FG% 46.0–47.5 %), so pooling all five seasons is reasonable. Rising FG% in 2022-23 and 2023-24 is worth remembering for the out-of-time test.

S.2 Sample

In [4]:
```
# Proportional stratified sample: preserves each season's share and its make/mis
frac = SAMPLE_SIZE / len(population)
sample = (population.groupby(["SEASON", TARGET], observed=True, group_keys=False
                    .sample(frac=frac, random_state=RANDOM_STATE).reset_index(dr

from scipy.stats import ks_2samp
check = pd.DataFrame({
    "population": [len(population), population[TARGET].mean(), population["SHOT_
                   (population["SHOT_TYPE"] == "3PT Field Goal").mean(), populat
    "sample": [len(sample), sample[TARGET].mean(), sample["SHOT_DISTANCE"].mean(
               (sample["SHOT_TYPE"] == "3PT Field Goal").mean(), sample["PERIOD"
    index=["shots", "FG%", "mean distance (ft)", "3PT share", "overtime share"])
check["KS p-value"] = ["", "", round(ks_2samp(population["SHOT_DISTANCE"], sampl
                       "", ""]
check.to_csv(REP / "sample_representativeness.csv"); display(check)
```

	population	sample	KS p-value
shots	1.031740e+06	199999.0000	
FG%	4.677000e-01	0.4677	
mean distance (ft)	1.356870e+01	13.5769	0.999
3PT share	3.917000e-01	0.3926	
overtime share	6.000000e-03	0.0059	

Representativeness. The 200k-shot sample (19 % of the population) reproduces the population almost exactly: FG% 46.77 % in both, mean distance 13.58 vs 13.57 ft, 3-point share 39.3 vs 39.2 %, overtime share 0.59 vs 0.60 %. The Kolmogorov–Smirnov test finds no difference in the shot-distance distribution (p = 0.999). Stratifying by **season** keeps each season's share; stratifying by **outcome** keeps the make rate exact. Because the sampling is *proportional*, the base rate is not distorted — important because the model must output real probabilities.

S.3 Data Partition

In [5]:
```
from sklearn.model_selection import GroupShuffleSplit

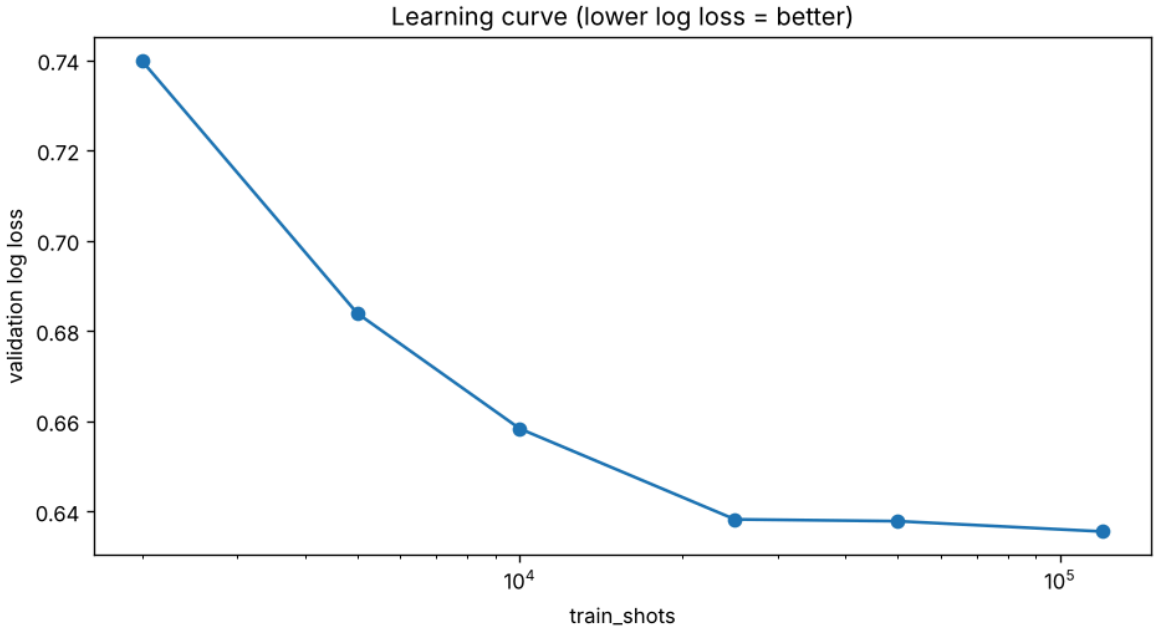
# Split by GAME_ID: shots from the same game never sit in two partitions (game-l
tr_frac, va_frac, te_frac = PARTITION
idx_train, idx_rest = next(GroupShuffleSplit(1, test_size=va_frac + te_frac, ran
                           .split(sample, groups=sample["GAME_ID"]))
rest = sample.iloc[idx_rest]
idx_val, idx_test = next(GroupShuffleSplit(1, test_size=te_frac / (va_frac + te_
                         .split(rest, groups=rest["GAME_ID"]))
train, valid, test = sample.iloc[idx_train], rest.iloc[idx_val], rest.iloc[idx_t
partitions = pd.DataFrame({p: {"shots": len(d), "games": d["GAME_ID"].nunique(),
                           for p, d in [("train", train), ("validate", valid), (
partitions.to_csv(REP / "data_partition.csv"); display(partitions)
assert not (set(train.GAME_ID) & set(valid.GAME_ID)) and not (set(train.GAME_ID)
```

	shots	games	FG%
train	120014.0	3497.0	0.4690
validate	40064.0	1166.0	0.4675
test	39921.0	1166.0	0.4639

In [6]:
```
# Is SAMPLE_SIZE big enough? Learning curve: validation performance vs. training
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss

quick = lambda d: d[["LOC_X", "LOC_Y", "PERIOD", "MINUTES_REMAINING", "SECONDS_R
    action=d["ACTION_TYPE"].cat.codes)
curve = []
for n in [2_000, 5_000, 10_000, 25_000, 50_000, len(train)]:
    sub = train.sample(n, random_state=RANDOM_STATE)
    m = HistGradientBoostingClassifier(max_iter=150, random_state=RANDOM_STATE).
    p = m.predict_proba(quick(valid))[:, 1]
    curve.append({"train_shots": n, "val_auc": roc_auc_score(valid[TARGET], p),
curve = pd.DataFrame(curve).round(4); curve.to_csv(REP / "learning_curve.csv", i
ax = curve.plot(x="train_shots", y="val_log_loss", marker="o", logx=True, legend
ax.set_ylabel("validation log loss"); savefig("sample_learning_curve")
```

	train_shots	val_auc	val_log_loss
0	2000	0.6122	0.7398
1	5000	0.6251	0.6839
2	10000	0.6401	0.6583
3	25000	0.6573	0.6382
4	50000	0.6561	0.6378
5	120014	0.6622	0.6355



Is the sample big enough? Validation log loss drops quickly up to ~25k training shots (0.740 → 0.638) and then flattens (0.6378 at 50k, 0.6355 at 120k). Doubling the data beyond that buys almost nothing, so modeling on the full 1 M shots would cost time without adding information — SEMMA's core argument for sampling. (The curve uses a quick feature set because Modify comes later; it is indicative, and the margin is wide.)

Partitioning. Train/validate/test = 60/20/20 **grouped by game** (3,497 / 1,166 / 1,166 games), so shots from one game never sit in two partitions and game-level effects (pace, arena, opponent) cannot leak. FG% is 46.9 / 46.8 / 46.4 %.

Output of Sample: a stratified, representative 200k-shot sample; three game-grouped partitions; plus the full 2024-25 season held back for an out-of-time check in Assess.

🔍 Critic review — Sample

Paste this phase (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Sample** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Sample step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether the sample is demonstrably representative, whether the sample size is justified, whether the partitioning prevents leakage (grouping by game, out-of-time hold-out), and whether stratification choices are explained. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Learning curve uses a reduced feature set, so it only indicates the needed size; stratifying on the target needs justification; two atypical seasons.	Stated as indicative (flat after ~25k shots, so 200k has a wide margin); showed proportional stratification keeps FG% exactly; checked season FG% (46.0–47.5 %).
2	`<fresh-chat critique>`	`<changes>`
3		

E — Explore

*Goal: search the data for trends, anomalies and relationships to gain understanding and ideas for Modify and Model. Explore uses the **training partition** only.*

E.1 StatExplore

In [7]:
```
display(train.describe(include="number").T.round(2))
cat_summary = pd.DataFrame({c: {"levels": train[c].nunique(), "most common": tra
                                "top share": round(train[c].value_counts(normali
                               for c in train.select_dtypes("category").columns}).T
display(cat_summary)

# Constant columns and outcome-revealing columns
constant = [c for c in train.columns if train[c].nunique() <= 1]
leak = pd.crosstab(train["EVENT_TYPE"], train[TARGET])
print("Constant columns:", constant); print("EVENT_TYPE vs target:"); display(le
```

	count	mean	std	min	25%
GAME_ID	120014.0	2.211121e+07	139540.70	2.190000e+07	2.200044e+07
GAME_EVENT_ID	120014.0	3.257700e+02	196.26	5.000000e+00	1.550000e+02
PLAYER_ID	120014.0	1.128043e+06	684217.25	1.713000e+03	2.035000e+05
TEAM_ID	120014.0	1.610613e+09	8.66	1.610613e+09	1.610613e+09
PERIOD	120014.0	2.490000e+00	1.13	1.000000e+00	1.000000e+00
MINUTES_REMAINING	120014.0	5.380000e+00	3.45	0.000000e+00	2.000000e+00
SECONDS_REMAINING	120014.0	2.887000e+01	17.43	0.000000e+00	1.400000e+01
SHOT_DISTANCE	120014.0	1.355000e+01	10.58	0.000000e+00	2.000000e+00
LOC_X	120014.0	-1.610000e+00	112.82	-2.500000e+02	-4.800000e+01
LOC_Y	120014.0	9.572000e+01	95.06	-5.000000e+01	1.400000e+01
SHOT_ATTEMPTED_FLAG	120014.0	1.000000e+00	0.00	1.000000e+00	1.000000e+00
SHOT_MADE_FLAG	120014.0	4.700000e-01	0.50	0.000000e+00	0.000000e+00
GAME_DATE	120014.0	2.021811e+07	14291.66	2.019102e+07	2.021022e+07

	levels	most common	top share
SEASON	5	2023-24	0.215
GRID_TYPE	1	Shot Chart Detail	1.0
PLAYER_NAME	896	Jayson Tatum	0.008
TEAM_NAME	30	Orlando Magic	0.037
EVENT_TYPE	2	Missed Shot	0.531
ACTION_TYPE	48	Jump Shot	0.316
SHOT_TYPE	2	2PT Field Goal	0.609
SHOT_ZONE_BASIC	7	Restricted Area	0.301
SHOT_ZONE_AREA	6	Center(C)	0.567
SHOT_ZONE_RANGE	5	Less Than 8 ft.	0.41
HTM	30	IND	0.037
VTM	30	ORL	0.038

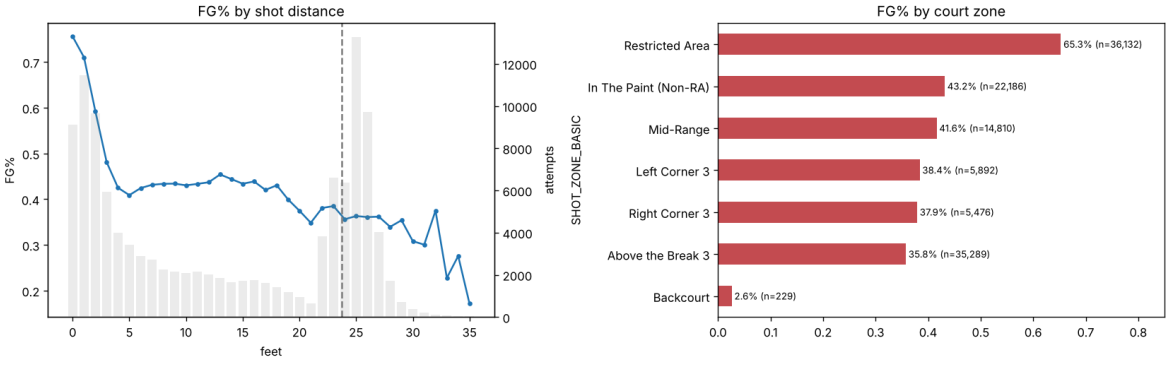
```
Constant columns: ['GRID_TYPE', 'SHOT_ATTEMPTED_FLAG']
EVENT_TYPE vs target:
```

SHOT_MADE_FLAG	**0**	**1**
EVENT_TYPE		
Made Shot	0	56283
Missed Shot	63731	0

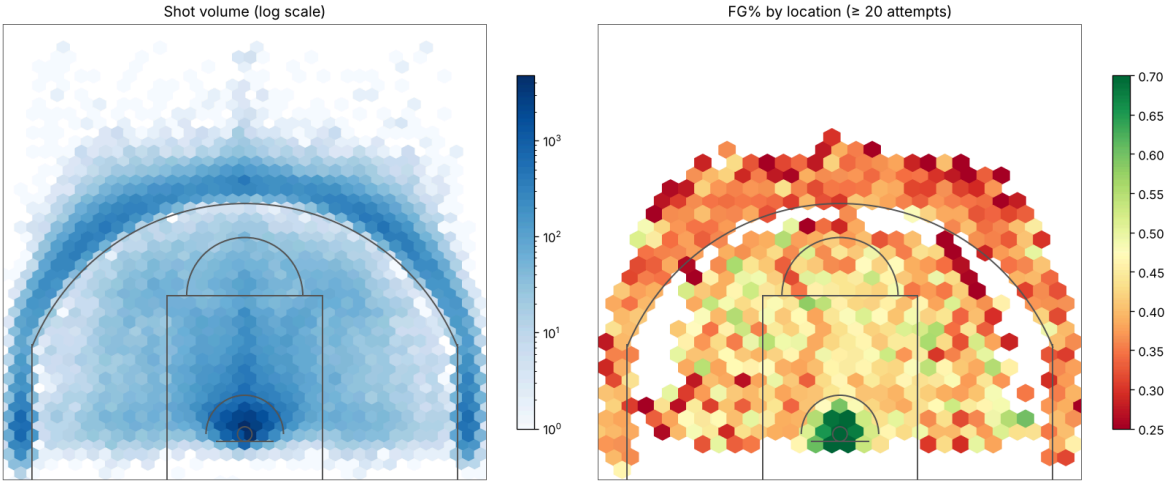
Findings. `EVENT_TYPE` ("Made Shot"/"Missed Shot") matches the target perfectly (56,283 makes, 63,731 misses, zero disagreements) — it **is the target in disguise** and must be dropped. `GRID_TYPE` and `SHOT_ATTEMPTED_FLAG` are constant. There are no missing values in any column. Shots cluster at the rim (30 % in the restricted area) and beyond the arc (39 % threes); `Jump Shot` alone is 32 % of attempts.

E.2 MultiPlot / Graph Explore

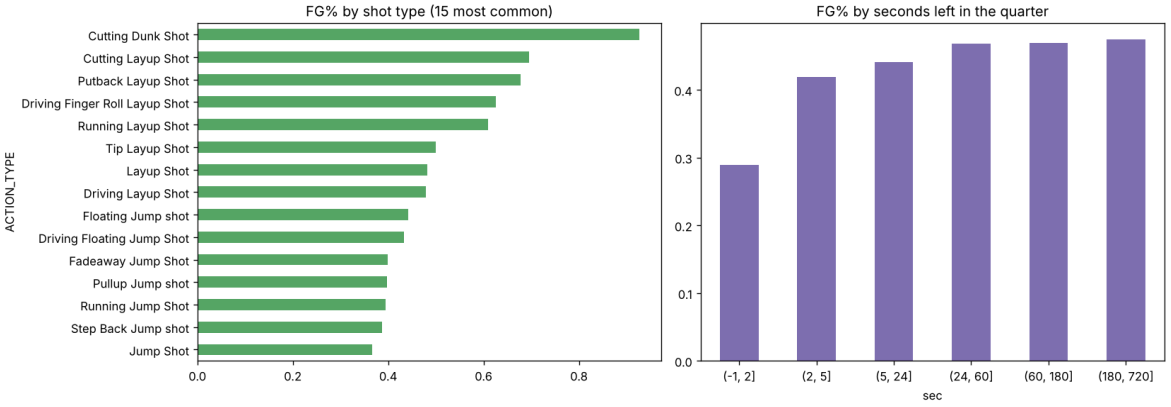
In [8]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
by_ft = train[train["SHOT_DISTANCE"] <= 35].groupby("SHOT_DISTANCE")[TARGET].agg
axes[0].plot(by_ft.index, by_ft["mean"], marker="o", ms=3); axes[0].axvline(23.7
axes[0].set(title="FG% by shot distance", xlabel="feet", ylabel="FG%")
ax2 = axes[0].twinx(); ax2.bar(by_ft.index, by_ft["size"], alpha=.15, color="gra
zone = train.groupby("SHOT_ZONE_BASIC", observed=True)[TARGET].agg(["mean", "siz
zone["mean"].plot.barh(ax=axes[1], color="#C44E52", title="FG% by court zone")
for i, (m, n) in enumerate(zip(zone["mean"], zone["size"])):
    axes[1].text(m, i, f" {m:.1%} (n={n:,})", va="center", fontsize=8)
axes[1].set_xlim(0, .85); savefig("explore_distance_zone")
```



In [9]:
```
# Shot chart: where shots are taken, and how often they go in
fig, axes = plt.subplots(1, 2, figsize=(14, 7))
h = train[train["LOC_Y"] < 400]
hb = axes[0].hexbin(h["LOC_X"], h["LOC_Y"], gridsize=40, bins="log", cmap="Blues
draw_court(axes[0]); axes[0].set_title("Shot volume (log scale)"); fig.colorbar(
hb2 = axes[1].hexbin(h["LOC_X"], h["LOC_Y"], C=h[TARGET], reduce_C_function=np.m
                     vmin=.25, vmax=.70, mincnt=20)
draw_court(axes[1]); axes[1].set_title("FG% by location (≥ 20 attempts)"); fig.c
savefig("explore_shot_charts")
```

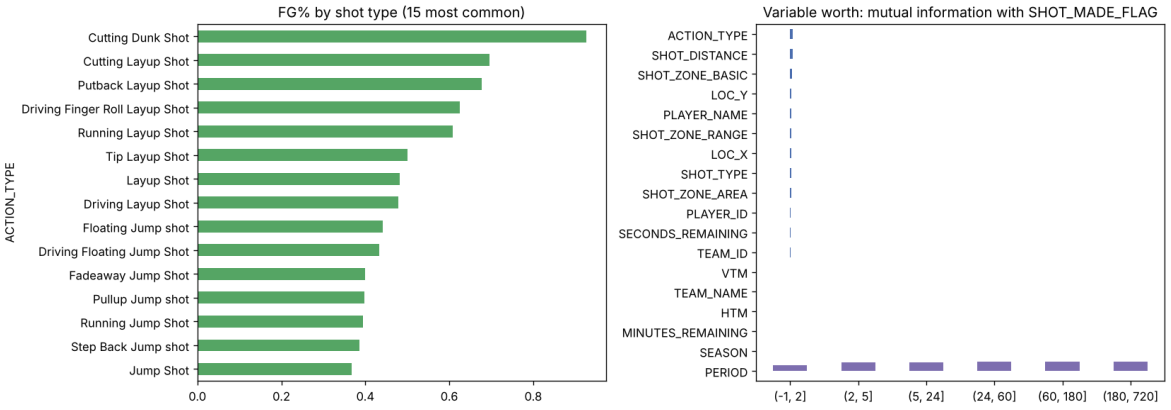


In [10]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
act = train.groupby("ACTION_TYPE", observed=True)[TARGET].agg(["mean", "size"]).
act["mean"].plot.barh(ax=axes[0], color="#55A868", title="FG% by shot type (15 m
clock = train.assign(sec=train["MINUTES_REMAINING"] * 60 + train["SECONDS_REMAIN
clock = clock[clock["PERIOD"] <= 4].groupby(pd.cut(clock["sec"], [-1, 2, 5, 24,
clock.plot.bar(ax=axes[1], rot=0, color="#8172B2", title="FG% by seconds left in
savefig("explore_action_clock")
```



E.3 Variable Selection (variable worth)

In [11]:
```
from sklearn.feature_selection import mutual_info_classif
cand = train.drop(columns=[TARGET, "EVENT_TYPE", "GRID_TYPE", "SHOT_ATTEMPTED_FL
s = cand.sample(30_000, random_state=RANDOM_STATE)
enc = s.apply(lambda c: c.cat.codes if hasattr(c, "cat") else c)
discrete = [hasattr(cand[c], "cat") or cand[c].nunique() < 20 for c in enc.colum
worth = pd.Series(mutual_info_classif(enc, train.loc[s.index, TARGET], discrete_
                  index=enc.columns).sort_values()
worth.plot.barh(color="#4C72B0", title="Variable worth: mutual information with
worth.sort_values(ascending=False).round(5).to_csv(REP / "variable_worth.csv")
```

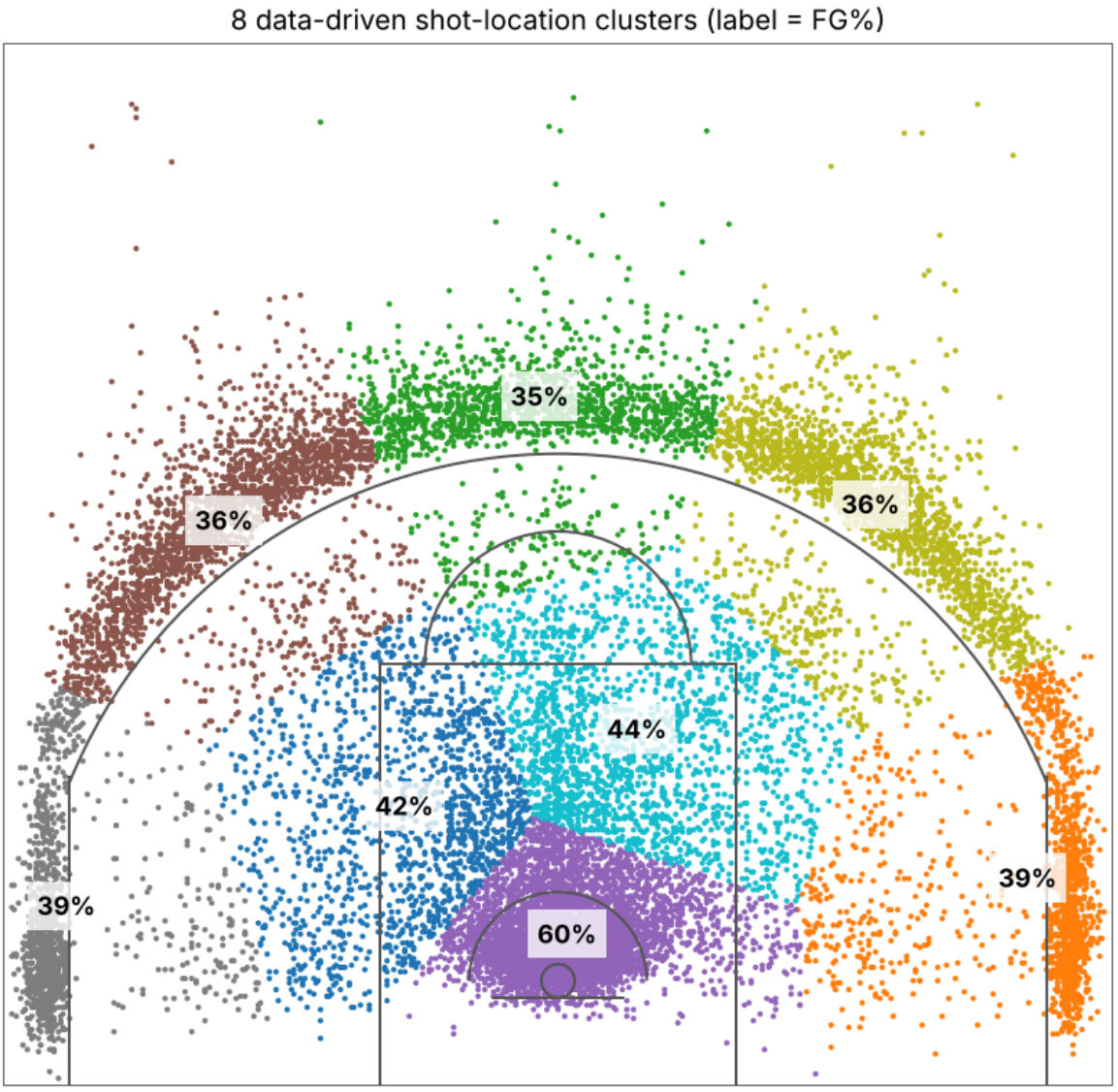


Variable worth. Shot type (`ACTION_TYPE` , MI 0.043) and distance (`SHOT_DISTANCE` 0.037, zones 0.030, `LOC_Y` 0.022) carry the most information. **Shooter identity also carries information** (`PLAYER_NAME` 0.022) but is excluded *by design*: this model measures shot **quality**; who made the shot is analysed afterwards (A.4) as shot **making**. Game-clock fields carry little on their own (`SECONDS_REMAINING` 0.004) but matter in a specific situation (end-of-quarter heaves, see E.2). Distance is represented several overlapping ways — Modify replaces them with one consistent geometry.

E.4 Cluster (shot-location segments)

In [12]:
```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=8, n_init=10, random_state=RANDOM_STATE).fit(train[["LOC_
seg = train.assign(cluster=km.labels_)
profile = seg.groupby("cluster")[TARGET].agg(attempts="size", fg_pct="mean").rou
profile["mean_distance_ft"] = seg.groupby("cluster")["SHOT_DISTANCE"].mean().rou
display(profile.sort_values("mean_distance_ft"))
fig, ax = plt.subplots(figsize=(7, 7)); h = seg[seg["LOC_Y"] < 400].sample(20_00
ax.scatter(h["LOC_X"], h["LOC_Y"], c=h["cluster"], cmap="tab10", s=2); draw_cour
for k, (x, y) in enumerate(km.cluster_centers_):
    ax.text(x, y, f"{profile.loc[k, 'fg_pct']:.0%}", ha="center", fontsize=11, w
            bbox=dict(facecolor="white", alpha=.8, edgecolor="none"))
ax.set_title("8 data-driven shot-location clusters (label = FG%)"); savefig("exp
```

cluster	attempts	fg_pct	mean_distance_ft
3	46323	0.602	2.2
0	8576	0.421	10.5
7	11489	0.444	11.6
1	8680	0.386	21.5
5	7809	0.388	22.2
4	13820	0.365	25.2
6	13305	0.358	25.2
2	10012	0.347	25.8



Exploration findings

#	Finding (training partition)	Implication for Modify / Model
1	FG% falls from 76 % at 0 ft to 43 % by 4–5 ft, stays flat (≈ 43 %) through mid-range, then drops to ≈ 36 % beyond the arc and 23 % at 30 ft	strongly non-linear distance effect → distance bins for the regression; trees handle it natively
2	Corner threes go in more often than above-the-break threes (38.1 % vs 35.6 %) — they are 22 ft instead of 23.75 ft	add a corner-three flag and a shot angle
3	Shot **type** matters as much as location: cutting layups 70 %, running layups 61 %, but plain layups only 48 % and jump shots 37 %	keep `ACTION_TYPE`; pool rare types
4	Shots with ≤ 2 s left in a quarter go in only 29 % of the time vs ≈ 47 % with time on the clock	add game-clock features and a "heave" flag
5	K-means on location finds one rim cluster (60 % FG, 2 ft), two mid-range clusters (42–44 %), two corner/short-three clusters (≈ 39 %) and three above-the-break clusters (35–37 %)	confirms that angle, not only distance, separates shot quality

#	Finding (training partition)	Implication for Modify / Model
⚠️	`EVENT_TYPE` duplicates the target; `GRID_TYPE`, `SHOT_ATTEMPTED_FLAG` constant	drop

🔍 Critic review — Explore

Paste this phase (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Explore** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Explore step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether exploration used only the training partition, whether findings are quantified and translated into concrete Modify actions, and whether leakage and redundant variables were detected. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Player identity has high variable worth but is dropped without argument; clusters are only descriptive.	Explained the shot-quality vs shot-making design choice; linked clusters to the angle/corner variables.
2	`<fresh-chat critique>`	`<changes>`
3		

M — Modify

*Goal: create, select and transform variables to focus the model-building process. Every transformation is written as **score code** — a small Python module reused unchanged by the deployed app, the equivalent of Enterprise Miner's exported score code.*

M.1 Drop

In [13]:
```
DROP = {
    "EVENT_TYPE": "duplicates the target (Made/Missed Shot) - leakage",
    "GRID_TYPE": "constant", "SHOT_ATTEMPTED_FLAG": "constant (every row is an a
    "GAME_ID": "identifier (used only for partitioning)", "GAME_EVENT_ID": "iden
```

```
    "PLAYER_ID": "shooter identity - excluded by design (model measures shot qua
    "TEAM_ID": "team identity - excluded by design", "TEAM_NAME": "team identity
    "SEASON": "season label - unseen seasons at scoring time",
    "SHOT_ZONE_BASIC": "derivable from LOC_X/LOC_Y - replaced by engineered geom
    "SHOT_ZONE_RANGE": "derivable from location", "SHOT_DISTANCE": "integer-roun
    "SHOT_TYPE": "2PT/3PT - recomputed from court geometry so the app needs only
}
drop_log = pd.DataFrame({"variable": list(DROP), "reason": list(DROP.values())})
drop_log.to_csv(REP / "modify_drop_log.csv", index=False); display(drop_log)
print("Inputs kept:", [c for c in train.columns if c not in DROP and c != TARGET
```

	variable	reason
0	EVENT_TYPE	duplicates the target (Made/Missed Shot) - lea...
1	GRID_TYPE	constant
2	SHOT_ATTEMPTED_FLAG	constant (every row is an attempt)
3	GAME_ID	identifier (used only for partitioning)
4	GAME_EVENT_ID	identifier
5	GAME_DATE	time index - would not generalise to new seasons
6	PLAYER_ID	shooter identity - excluded by design (model m...
7	PLAYER_NAME	shooter identity - excluded by design
8	TEAM_ID	team identity - excluded by design
9	TEAM_NAME	team identity - excluded by design
10	HTM	home team code
11	VTM	visiting team code
12	SEASON	season label - unseen seasons at scoring time
13	SHOT_ZONE_BASIC	derivable from LOC_X/LOC_Y - replaced by engin...
14	SHOT_ZONE_AREA	derivable from location
15	SHOT_ZONE_RANGE	derivable from location
16	SHOT_DISTANCE	integer-rounded; replaced by exact distance fr...
17	SHOT_TYPE	2PT/3PT - recomputed from court geometry so th...

```
Inputs kept: ['PERIOD', 'MINUTES_REMAINING', 'SECONDS_REMAINING', 'ACTION_TYPE',
'LOC_X', 'LOC_Y']
```

M.2 Transform Variables · Replacement · Interactive Binning — as score code

In [14]:

```
SCORE_CODE = r"""
# Score code for the NBA xFG% model: the Modify step, shared by the notebook and
import numpy as np
import pandas as pd

RAW_INPUTS = ["LOC_X", "LOC_Y", "ACTION_TYPE", "PERIOD", "MINUTES_REMAINING", "S
NUMERIC = ["dist_ft", "angle_deg", "abs_angle", "is_three", "is_corner_three", "
```

```
            "PERIOD", "is_overtime", "clutch", "heave"]
CATEGORICAL = ["action", "distance_bin"]
BIN_EDGES = [-0.1, 4, 10, 16, 23.75, 30, 100]
BIN_LABELS = ["0-4 ft", "4-10 ft", "10-16 ft", "16-23.75 ft", "23.75-30 ft", "30


def engineer(raw):
    # Transform raw shot records (RAW_INPUTS) into model inputs. Stateless, so i
    d = pd.DataFrame(raw).copy()
    x, y = d["LOC_X"].astype(float), d["LOC_Y"].astype(float)       # tenths of
    out = pd.DataFrame(index=d.index)
    out["dist_ft"] = np.hypot(x, y) / 10
    out["angle_deg"] = np.degrees(np.arctan2(x, y))                  # 0 = strai
    out["abs_angle"] = out["angle_deg"].abs()
    corner = (x.abs() >= 220) & (y <= 92.5)
    out["is_corner_three"] = corner.astype(int)
    out["is_three"] = (corner | (np.hypot(x, y) >= 237.5)).astype(int)
    out["secs_left_period"] = d["MINUTES_REMAINING"].astype(int) * 60 + d["SECON
    out["PERIOD"] = d["PERIOD"].astype(int)
    out["is_overtime"] = (out["PERIOD"] > 4).astype(int)
    out["clutch"] = ((out["PERIOD"] >= 4) & (out["secs_left_period"] <= 120)).as
    out["heave"] = ((out["secs_left_period"] <= 3) & (out["dist_ft"] >= 30)).ast
    out["action"] = d["ACTION_TYPE"].astype(str)
    out["distance_bin"] = pd.cut(out["dist_ft"], BIN_EDGES, labels=BIN_LABELS).a
    return out[NUMERIC + CATEGORICAL]


class AveragedModel:
    # Ensemble node: averages the predicted probabilities of already-fitted pipe
    def __init__(self, models):
        self.models = models
        self.classes_ = np.array([0, 1])

    def predict_proba(self, X):
        p = np.mean([m.predict_proba(X)[:, 1] for m in self.models], axis=0)
        return np.column_stack([1 - p, p])

    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)
"""
(APP / "shot_features.py").write_text(SCORE_CODE.lstrip())
if str(APP) not in sys.path:
    sys.path.insert(0, str(APP))
import importlib, shot_features
importlib.reload(shot_features)
from shot_features import engineer, AveragedModel, RAW_INPUTS, NUMERIC, CATEGORI

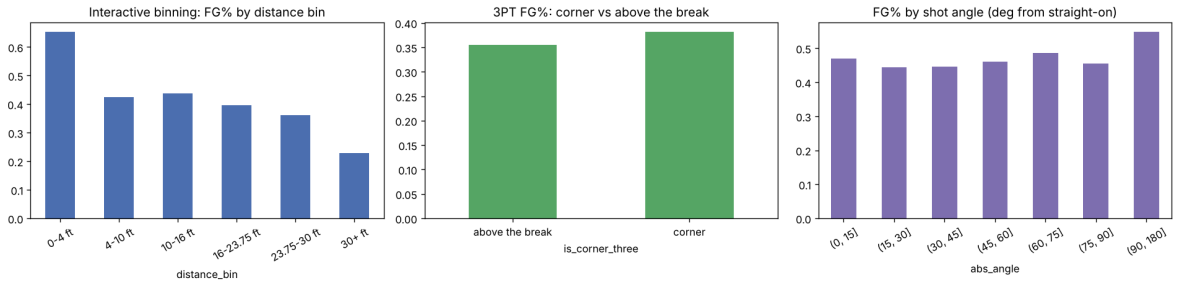
agree = (engineer(train)["is_three"] == (train["SHOT_TYPE"] == "3PT Field Goal")
print(f"Geometry-based 3PT flag agrees with the official SHOT_TYPE on {agree:.2%
engineer(train.head()).join(train[["SHOT_TYPE", "SHOT_DISTANCE", TARGET]].head()
```

```
Geometry-based 3PT flag agrees with the official SHOT_TYPE on 99.95% of training
shots
```

Out[14]:

	dist_ft	angle_deg	abs_angle	is_three	is_corner_three	secs_left_period	PERIOD
0	26.482069	30.147421	30.147421	1	0	398	3
1	26.126806	-42.052414	42.052414	1	0	641	2
2	24.647921	48.289243	48.289243	1	0	205	3
4	26.447306	31.706851	31.706851	1	0	548	4
5	19.089526	30.191623	30.191623	0	0	13	4

In [15]:
```
# Did the transformations create signal? FG% by the new variables (training part
eng = engineer(train).assign(made=train[TARGET].to_numpy())
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
eng.groupby("distance_bin")["made"].mean().reindex(shot_features.BIN_LABELS).plo
eng[eng["is_three"] == 1].groupby("is_corner_three")["made"].mean().rename({0: "
eng.groupby(pd.cut(eng["abs_angle"], [0, 15, 30, 45, 60, 75, 90, 180]), observed
savefig("modify_new_variables")
```



M.3 Replacement & Impute (inside the model pipeline)

*Rare `action` levels (< 0.5 % of training shots) are **replaced** by a pooled "infrequent" level; missing values (none in training, but possible in new data) are **imputed** — both learned from the training partition only, inside the pipeline.*

In [16]:
```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardSc
from sklearn.impute import SimpleImputer

def make_prep():
    return ColumnTransformer([
        ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale"
        ("cat", Pipeline([("impute", SimpleImputer(strategy="constant", fill_val
                          ("replace_rare", OneHotEncoder(handle_unknown="infrequ
```

```
prep_preview = make_prep().fit(engineer(train))
names = prep_preview.get_feature_names_out()
print(f"{len(names)} model inputs after Replacement/one-hot; pooled rare actions
      sorted(prep_preview.named_transformers_["cat"].named_steps["replace_rare"]
X_tr, X_va, X_te = (d[RAW_INPUTS] for d in (train, valid, test))
y_tr, y_va, y_te = (d[TARGET].to_numpy() for d in (train, valid, test))
```

```
46 model inputs after Replacement/one-hot; pooled rare actions: ['Alley Oop Layup
shot', 'Cutting Finger Roll Layup Shot', 'Driving Bank Hook Shot', 'Driving Rever
se Dunk Shot', 'Fadeaway Bank shot', 'Finger Roll Layup Shot', 'Hook Bank Shot',
'Jump Bank Shot', 'Putback Dunk Shot', 'Reverse Dunk Shot', 'Running Alley Oop Du
nk Shot', 'Running Alley Oop Layup Shot'] ...
```

🔍 Critic review — Modify

Paste this phase (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Modify** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Modify step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether every transformation is justified by an Explore finding, whether anything is fitted on non-training data, whether dropped variables (leakage, identity, redundancy) are correctly reasoned, and whether the score code is reusable at deployment. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	`ACTION_TYPE` is coded by scorers and could reflect the outcome; verify the geometry-based 3PT flag.	Recorded as a limitation (standard in public xFG models); showed 99.95 % agreement of the 3PT flag.
2	`<fresh-chat critique>`	`<changes>`
3		

M — Model

*Goal: let the software search for a combination of inputs that reliably predicts the outcome. Several model families compete; the **validation** partition is used to tune and compare them (as Enterprise Miner does).*

In [17]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import brier_score_loss, accuracy_score

pipe = lambda est: Pipeline([("modify", FunctionTransformer(engineer)), ("prep",
def scores(y, p):
    return {"log_loss": log_loss(y, p), "brier": brier_score_loss(y, p), "roc_au

fitted, val_pred, fit_secs = {}, {}, {}
```

Baseline — zone average

In [18]:
```
# The naive analyst's model: every shot gets the league FG% of its official cour
zone_keys = ["SHOT_ZONE_BASIC", "SHOT_ZONE_AREA", "SHOT_ZONE_RANGE"]
zone_rate = train.groupby(zone_keys, observed=True)[TARGET].mean()
def zone_baseline(d):
    return d[zone_keys].astype(str).apply(tuple, axis=1).map({tuple(map(str, k))
val_pred["Zone average (baseline)"] = zone_baseline(valid)
print("Baseline validation:", {k: round(v, 4) for k, v in scores(y_va, val_pred[
```

```
Baseline validation: {'log_loss': 0.6572, 'brier': 0.2324, 'roc_auc': 0.6341, 'ac
curacy': 0.6274}
```

Regression node (logistic regression)

In [19]:
```
t = time.time(); fitted["Regression"] = pipe(LogisticRegression(max_iter=2000)).
val_pred["Regression"] = fitted["Regression"].predict_proba(X_va)[:, 1]
coef = pd.Series(fitted["Regression"].named_steps["model"].coef_[0], index=fitte
display(coef.sort_values().iloc[np.r_[0:6, -6:0]].round(3).to_frame("coefficient
```

	coefficient (log-odds)
cat__action_Tip Layup Shot	-0.999
num__dist_ft	-0.880
cat__action_Layup Shot	-0.783
cat__action_Driving Layup Shot	-0.765
cat__distance_bin_4-10 ft	-0.584
cat__action_Driving Reverse Layup Shot	-0.497
cat__distance_bin_30+ ft	0.495
cat__action_Driving Dunk Shot	0.510
cat__action_Dunk Shot	1.029
cat__action_Alley Oop Dunk Shot	1.282
cat__action_Cutting Dunk Shot	1.650
cat__action_Running Dunk Shot	1.705

Decision Tree node (depth chosen on the validation partition)

In [20]:
```
tree_search = []
for depth in [4, 6, 8, 10, 12, 15]:
    m = pipe(DecisionTreeClassifier(max_depth=depth, min_samples_leaf=100, rando
    tree_search.append({"max_depth": depth, "val_log_loss": log_loss(y_va, m.pre
tree_search = pd.DataFrame(tree_search)
best = tree_search.loc[tree_search["val_log_loss"].idxmin()]
fitted["Decision Tree"] = best["model"]; fit_secs["Decision Tree"] = np.nan
val_pred["Decision Tree"] = fitted["Decision Tree"].predict_proba(X_va)[:, 1]
display(tree_search.drop(columns="model").round(4)); print("Chosen depth (lowest
```

	max_depth	val_log_loss
0	4	0.6427
1	6	0.6400
2	8	0.6399
3	10	0.6430
4	12	0.6430
5	15	0.6454

```
Chosen depth (lowest validation log loss): 8
```

Neural Network node

In [21]:
```
t = time.time()
fitted["Neural Network"] = pipe(MLPClassifier(hidden_layer_sizes=(32, 16), alpha
                                              random_state=RANDOM_STATE)).fit(X_
fit_secs["Neural Network"] = time.time() - t
val_pred["Neural Network"] = fitted["Neural Network"].predict_proba(X_va)[:, 1]
print(f"Stopped after {fitted['Neural Network'].named_steps['model'].n_iter_} ep
```

```
Stopped after 37 epochs (11s)
```

Gradient Boosting node

In [22]:
```
# Critique fix: tune the boosting model on the validation partition (as the tree
gb_search = []
for lr, leaves in [(0.05, 15), (0.05, 31), (0.08, 31), (0.10, 63)]:
    t = time.time()
    m = pipe(HistGradientBoostingClassifier(learning_rate=lr, max_iter=500, max_
                                            early_stopping=True, random_state=RA
    gb_search.append({"learning_rate": lr, "max_leaf_nodes": leaves, "iterations
                      "val_log_loss": log_loss(y_va, m.predict_proba(X_va)[:, 1]
gb_search = pd.DataFrame(gb_search)
best_gb = gb_search.loc[gb_search["val_log_loss"].idxmin()]
fitted["Gradient Boosting"], fit_secs["Gradient Boosting"] = best_gb["model"], b
val_pred["Gradient Boosting"] = fitted["Gradient Boosting"].predict_proba(X_va)[
gb_search.drop(columns="model").round(4).to_csv(REP / "gradient_boosting_tuning.
display(gb_search.drop(columns="model").round(4))
print(f"Chosen: learning_rate={best_gb['learning_rate']}, max_leaf_nodes={best_g
```

	learning_rate	max_leaf_nodes	iterations	val_log_loss	seconds
0	0.05	15	182	0.6346	3.8851
1	0.05	31	165	0.6346	3.8969
2	0.08	31	108	0.6348	2.8717
3	0.10	63	65	0.6352	2.4487

```
Chosen: learning_rate=0.05, max_leaf_nodes=31
```

Ensemble node

In [23]:
```
fitted["Ensemble"] = AveragedModel([fitted[m] for m in ["Regression", "Neural Ne
val_pred["Ensemble"] = fitted["Ensemble"].predict_proba(X_va)[:, 1]
print("Ensemble = average of Regression, Neural Network and Gradient Boosting pr
```

```
Ensemble = average of Regression, Neural Network and Gradient Boosting probabilit
ies
```

Model families in the competition

Node	What it is	Assumptions / strengths	Settings
Zone average	league FG% of the official zone	no learning beyond the zone; the "analyst's" benchmark	—
Regression	logistic regression on scaled inputs + one-hot shot type and distance bins	linear in log-odds — distance bins supply the non-linearity; fully interpretable (dunks +1.0 to +1.7 log-odds, tip layups −1.0)	default L2, `max_iter=2000`
Decision Tree	single CART tree	captures interactions, easy to read; high variance	depth tuned on validation: 8 (log loss 0.6399)
Neural Network	MLP, 32-16 hidden units	flexible smooth functions; needs scaled inputs	early stopping (37 epochs)
Gradient Boosting	histogram gradient-boosted trees	strongest on tabular data; interactions + non-linearity	tuned on validation over 4 settings: learning rate 0.05, 31 leaves, 165 iterations
Ensemble	average of regression, NN and boosting probabilities	reduces variance if the members disagree	—

🔍 Critic review — Model

Paste this phase (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Model** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Model step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether the competing model families are well chosen and fairly configured, whether tuning used only the validation partition, whether a meaningful baseline is included, and whether settings are documented. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Unfair comparison — the tree was tuned on validation but boosting was not; neural network untuned.	Tuned boosting over 4 settings on validation (lr 0.05, 31 leaves); NN left as is (ensemble including it does not beat boosting).
2	`<fresh-chat critique>`	`<changes>`
3		

A — Assess

Goal: evaluate the usefulness and reliability of the findings — compare models, check calibration, confirm on untouched data, and score new data.

A.1 Model Comparison (validation partition)

In [24]:
```
comparison = pd.DataFrame({m: scores(y_va, p) for m, p in val_pred.items()}).T
comparison["fit_seconds"] = pd.Series(fit_secs).round(1)
comparison = comparison.sort_values("log_loss").round(4)
comparison.to_csv(REP / "model_comparison_validation.csv"); display(comparison)
CHAMPION = comparison.drop(index="Zone average (baseline)").index[0]
beats_baseline = comparison.loc[CHAMPION, "log_loss"] < comparison.loc["Zone ave
print(f"Champion (lowest validation log loss): {CHAMPION} | beats zone baseline:
```

	log_loss	brier	roc_auc	accuracy	fit_seconds
Gradient Boosting	0.6346	0.2229	0.6646	0.6355	3.9
Ensemble	0.6352	0.2232	0.6644	0.6367	NaN
Neural Network	0.6376	0.2242	0.6579	0.6329	11.1
Decision Tree	0.6399	0.2251	0.6575	0.6310	NaN
Regression	0.6400	0.2253	0.6590	0.6310	1.4
Zone average (baseline)	0.6572	0.2324	0.6341	0.6274	NaN

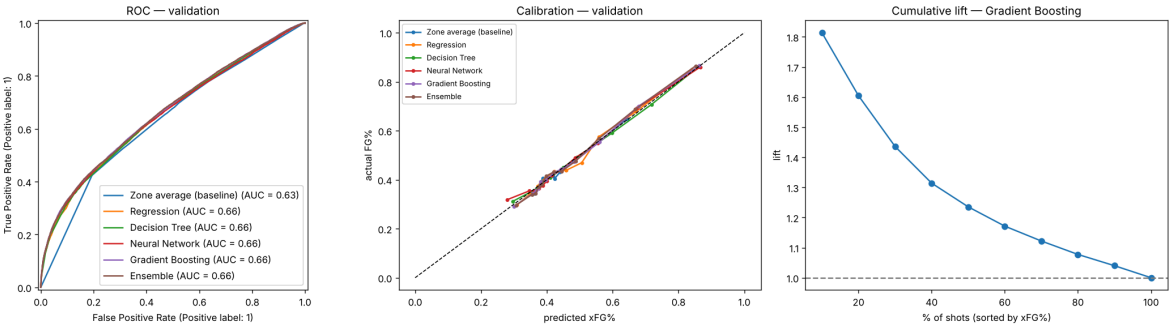
```
Champion (lowest validation log loss): Gradient Boosting | beats zone baseline: T
rue
```

Why log loss? xFG% is used as a **probability** (expected points = xFG% × shot value), so the selection criterion must reward accurate probabilities — not just ranking (AUC) or a 50 % cutoff (accuracy). Brier score agrees with log loss on the ranking of every model.

Result. Gradient boosting wins (log loss 0.6346), narrowly ahead of the ensemble (0.6352) and the neural network (0.6376). Every learned model beats the zone baseline (0.6572) — a 3.4 % reduction in log loss and AUC 0.665 vs 0.634. The gain looks modest because most of a shot's outcome is genuinely random once you know only *where* and *what* it was; the model cannot see the defender, the shooter's balance or the pass. All models are well calibrated on validation, and the champion's cumulative lift chart shows its top-rated 10 % of shots go in ≈ 1.8× as often as average.

In [25]:
```
from sklearn.metrics import RocCurveDisplay
from sklearn.calibration import calibration_curve
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for m, p in val_pred.items():
    RocCurveDisplay.from_predictions(y_va, p, ax=axes[0], name=m)
    frac_pos, mean_pred = calibration_curve(y_va, p, n_bins=12, strategy="quanti
    axes[1].plot(mean_pred, frac_pos, marker="o", ms=3, label=m)
axes[0].set_title("ROC — validation"); axes[1].plot([0, 1], [0, 1], "k--", lw=1)
axes[1].set(title="Calibration — validation", xlabel="predicted xFG%", ylabel="a

# Cumulative lift chart (Enterprise Miner's standard assessment plot)
order = np.argsort(-val_pred[CHAMPION]); base = y_va.mean()
deciles = np.array_split(y_va[order], 10)
cum_lift = np.cumsum([d.sum() for d in deciles]) / np.cumsum([len(d) for d in de
axes[2].plot(range(10, 101, 10), cum_lift, marker="o"); axes[2].axhline(1, c="gr
axes[2].set(title=f"Cumulative lift — {CHAMPION}", xlabel="% of shots (sorted by
savefig("assess_model_comparison")
```



A.2 Cutoff (how predictions translate into make/miss calls)

In [26]:
```
p = val_pred[CHAMPION]
cutoff = pd.DataFrame([{"cutoff": c, "predicted_makes_%": (p >= c).mean() * 100,
                        "precision": y_va[p >= c].mean() if (p >= c).any() else
                        "recall": (y_va[p >= c].sum() / y_va.sum())} for c in [0
cutoff.to_csv(REP / "assess_cutoff.csv", index=False); display(cutoff)
```

	cutoff	predicted_makes_%	precision	recall
0	0.3	97.122	0.475	0.986
1	0.4	52.890	0.568	0.643
2	0.5	27.209	0.689	0.401
3	0.6	17.914	0.771	0.295
4	0.7	11.185	0.834	0.199

xFG% is used as a probability, so no single cutoff is required. The table shows what labels would mean: shots rated ≥ 60 % go in 77 % of the time (17.9 % of shots), and ≥ 70 % go in 83 % of the time — "high-quality shot" labels are reliable.

A.3 Final check on untouched data — test partition and the unseen 2024-25 season

In [27]:
```
oot = load_season(OUT_OF_TIME_SEASON)
final = {
    "validate (2019-24 sample)": (y_va, fitted[CHAMPION].predict_proba(X_va)[:,
    "test (2019-24 sample)": (y_te, fitted[CHAMPION].predict_proba(X_te)[:, 1],
    f"out-of-time ({oot['SEASON'].iloc[0]}, all {len(oot):,} shots)": (oot[TARGE
}
final_table = pd.DataFrame({k: {**scores(y, p), "baseline_log_loss": log_loss(y,
                               for k, (y, p, b) in final.items()}).T.round(4)
final_table.to_csv(REP / "assess_final_performance.csv"); display(final_table)
```

	log_loss	brier	roc_auc	accuracy	baseline_log_loss	mean_xFG	actual_FG
validate (2019-24 sample)	0.6346	0.2229	0.6646	0.6355	0.6572	0.4691	0.4675
test (2019-24 sample)	0.6345	0.2231	0.6626	0.6329	0.6583	0.4686	0.4639
out-of-time (2024-25, all 219,527 shots)	0.6377	0.2244	0.6574	0.6300	0.6577	0.4643	0.4672

In [28]:
```
# Critique fix: calibration by decile on the unseen 2024-25 season (not only on
y_oot, p_oot = final[list(final)[-1]][:2]
cal = pd.DataFrame({"xFG": p_oot, "made": y_oot})
cal["decile"] = pd.qcut(cal["xFG"], 10, labels=[f"D{i}" for i in range(1, 11)])
cal = cal.groupby("decile", observed=True).agg(shots=("made", "size"), mean_xFG=
cal["gap_pct_points"] = (cal["actual_FG"] - cal["mean_xFG"]) * 100
cal = cal.round(3); cal.to_csv(REP / "calibration_by_decile_2024_25.csv"); displ
print(f"Largest decile gap: {cal['gap_pct_points'].abs().max():.1f} percentage p
```

decile	shots	mean_xFG	actual_FG	gap_pct_points
D1	21958	0.305	0.310	0.454
D2	21955	0.359	0.358	-0.112
D3	21954	0.370	0.366	-0.395
D4	21945	0.378	0.377	-0.059
D5	21952	0.393	0.391	-0.183
D6	21952	0.423	0.430	0.699
D7	21953	0.447	0.455	0.734
D8	21957	0.504	0.508	0.319
D9	21950	0.627	0.632	0.522
D10	21951	0.837	0.846	0.885

```
Largest decile gap: 0.9 percentage points
```

No over-fitting, and it travels to a new season. Test log loss (0.6345) equals validation (0.6346), so choosing the champion on validation did not over-fit. On the full 2024-25 season, which the model never saw, log loss is 0.6377 (baseline 0.6577) and AUC 0.657 — a small, expected decline. Calibration-in-the-large is excellent: predicted 46.4 % vs actual 46.7 %, and by decile every gap is ≤ 0.9 percentage points (from 30.5 % predicted / 31.0 % actual in the lowest decile to 83.7 % / 84.6 % in the highest). The three success criteria are met.

A.4 Score & Segment Profile — shot-making over expected (2024-25)

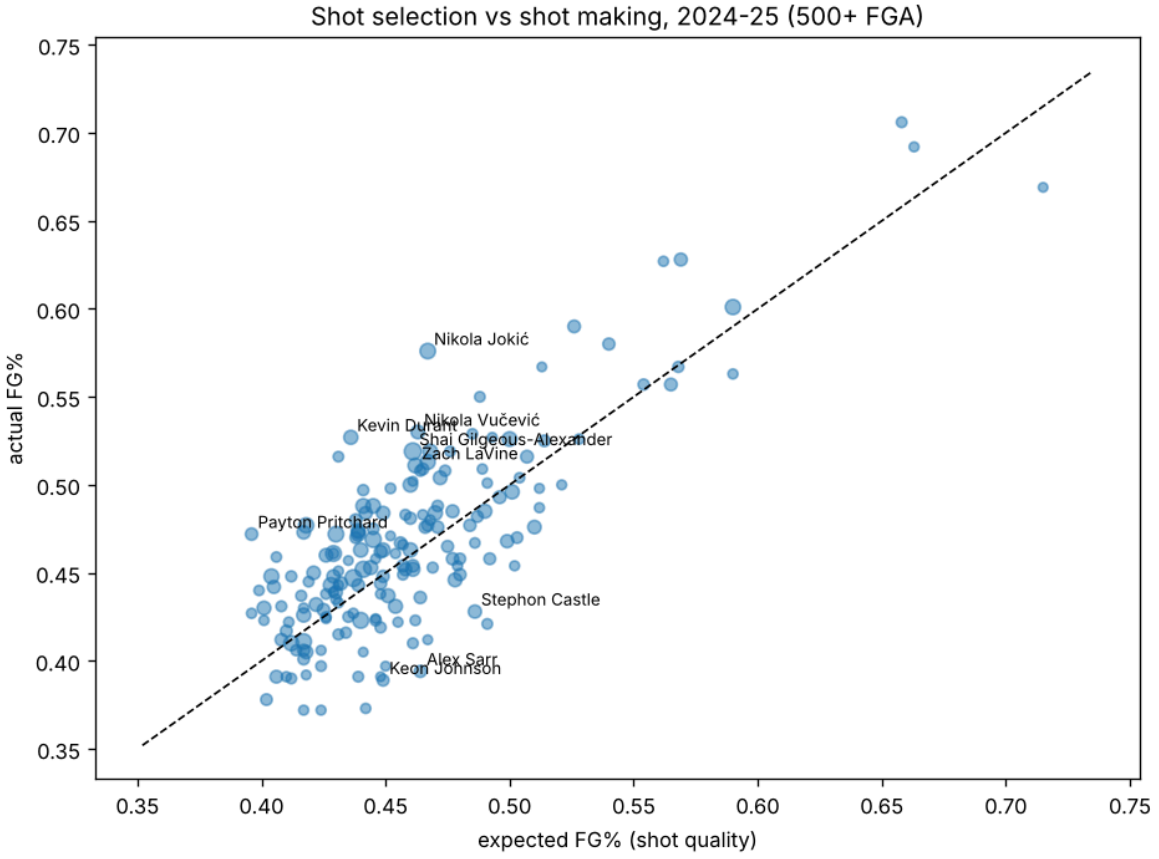
In [29]:
```
oot = oot.assign(xFG=final[list(final)[-1]][1], pts=np.where(engineer(oot)["is_t
players = oot.groupby("PLAYER_NAME", observed=True).agg(team=("TEAM_NAME", lambd
                                                        FG_pct=(TARGET, "mean")
                                                        points=(TARGET, lambda
                                                        expected_points=("xFG",
players = players[players["FGA"] >= 500].assign(FG_over_expected=lambda d: d["FG
                                                points_over_expected=lambda d: d
players = players.sort_values("points_over_expected", ascending=False).round(3)
players.to_csv(REP / "shot_making_leaderboard_2024_25.csv")
print(f"{len(players)} players with 500+ FGA in {oot['SEASON'].iloc[0]}")
display(players.head(15)); display(players.tail(5))
```

181 players with 500+ FGA in 2024-25

PLAYER_NAME	team	FGA	FG_pct	xFG_pct	points	expected_points	FG_over_exp
Nikola Jokić	Denver Nuggets	1364	0.576	0.467	1710	1387.082	
Kevin Durant	Phoenix Suns	1124	0.527	0.436	1344	1116.066	
Shai Gilgeous-Alexander	Oklahoma City Thunder	1656	0.519	0.461	1883	1685.543	
Payton Pritchard	Boston Celtics	866	0.472	0.396	1073	908.481	
Zach LaVine	Chicago Bulls	1223	0.511	0.462	1489	1326.119	
Nikola Vučević	Chicago Bulls	1038	0.530	0.463	1229	1078.253	
DeMar DeRozan	Sacramento Kings	1308	0.477	0.418	1332	1184.790	
Stephen Curry	Golden State Warriors	1258	0.448	0.404	1439	1295.851	
Tyrese Haliburton	Indiana Pacers	1006	0.473	0.417	1170	1040.840	
Ty Jerome	Cleveland Cavaliers	618	0.516	0.431	749	622.189	
Tyler Herro	Miami Heat	1378	0.472	0.430	1553	1426.306	
Pascal Siakam	Indiana Pacers	1182	0.519	0.468	1352	1225.433	
LeBron James	Los Angeles Lakers	1270	0.513	0.467	1451	1328.579	
Domantas Sabonis	Sacramento Kings	893	0.590	0.526	1119	996.732	
Jaren Jackson Jr.	Memphis Grizzlies	1213	0.488	0.441	1330	1210.549	

PLAYER_NAME	team	FGA	FG_pct	xFG_pct	points	expected_points	FG_over_exp
Scottie Barnes	Toronto Raptors	1063	0.446	0.478	1024	1117.643	-
Bilal Coulibaly	Washington Wizards	624	0.421	0.491	589	695.528	-
Keon Johnson	Brooklyn Nets	779	0.389	0.449	732	844.878	-
Alex Sarr	Washington Wizards	828	0.394	0.464	757	894.239	-
Stephon Castle	San Antonio Spurs	988	0.428	0.486	941	1078.673	-

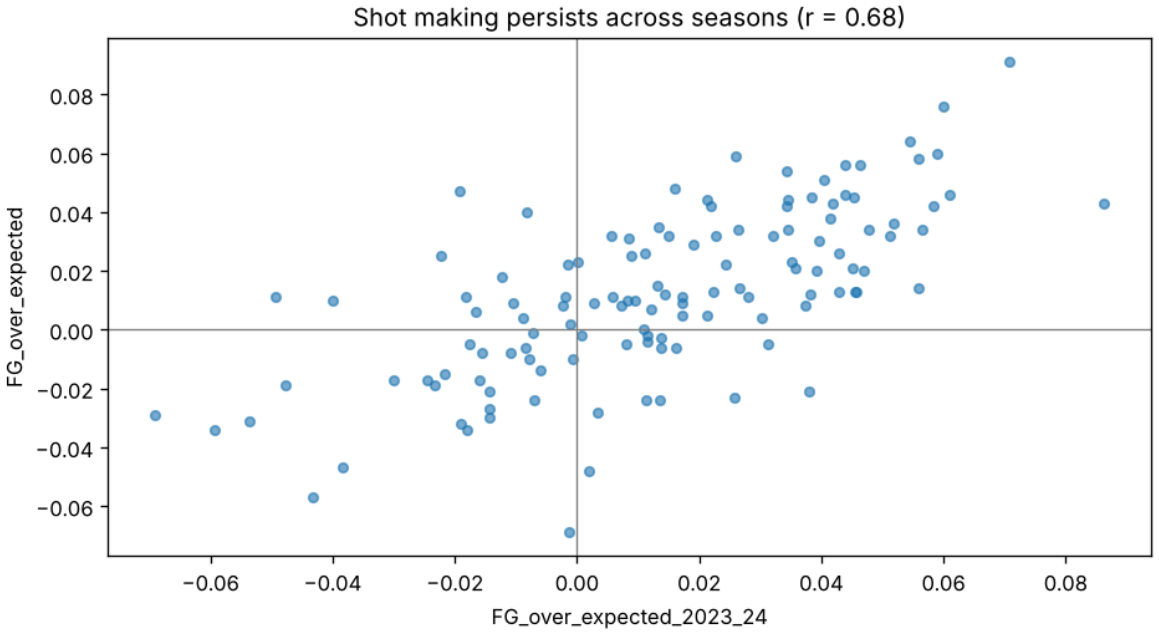
In [30]:
```
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(players["xFG_pct"], players["FG_pct"], s=players["FGA"] / 25, alpha=.
lim = [players[["xFG_pct", "FG_pct"]].min().min() - .02, players[["xFG_pct", "FG
ax.plot(lim, lim, "k--", lw=1)
for name, r in pd.concat([players.head(6), players.tail(3)]).iterrows():
    ax.annotate(name, (r["xFG_pct"], r["FG_pct"]), fontsize=8, xytext=(3, 3), te
ax.set(xlabel="expected FG% (shot quality)", ylabel="actual FG%", title="Shot se
savefig("assess_shot_making")
```



In [31]:
```
# Critique fix: is "shot making over expected" a repeatable skill or noise? Comp
prev = population[population["SEASON"] == "2023-24"]
prev = prev.assign(xFG=fitted[CHAMPION].predict_proba(prev[RAW_INPUTS])[:, 1])
prev_players = prev.groupby("PLAYER_NAME", observed=True).agg(FGA=(TARGET, "size
```

```
prev_players = prev_players[prev_players["FGA"] >= 500]
prev_players.index = prev_players.index.astype(str)
both = (players[["FG_over_expected"]].set_axis(players.index.astype(str))
        .join((prev_players["FG"] - prev_players["xFG"]).rename("FG_over_expecte
r = both.corr().iloc[0, 1]
both.to_csv(REP / "shot_making_persistence.csv")
print(f"{len(both)} players with 500+ FGA in both seasons; season-to-season corr
ax = both.plot.scatter("FG_over_expected_2023_24", "FG_over_expected", alpha=.6,
ax.axhline(0, c="gray", lw=.8); ax.axvline(0, c="gray", lw=.8); savefig("assess_
```

```
124 players with 500+ FGA in both seasons; season-to-season correlation of FG ove
r expected: r = 0.68
```



Reading the chart: above the diagonal = the player makes more than his shots would predict (**shot making**); further right = he takes higher-quality shots (**shot selection**).

- **Nikola Jokić** leads by far: 57.6 % on shots worth 46.7 % — **+323 points over expected** in one season. His shots are not unusually easy; he simply converts them.
- **Kevin Durant** (+228) and **Shai Gilgeous-Alexander** (+197) take *harder* shots than Jokić (xFG 43.6 % and 46.1 % vs 46.7 %) and still make them — the profile of elite scorers.
- **Stephen Curry**'s xFG% is low (40.4 %) because of his shot diet (long threes), yet he is +143: raw FG% (44.8 %) badly undersells him.
- At the bottom are mostly young players (Stephon Castle, Alex Sarr, Bilal Coulibaly; −107 to −138 points), consistent with shooting development curves.

Is this a skill or noise? For 124 players with 500+ attempts in both 2023-24 and 2024-25, FG over expected correlates **r = 0.68** across seasons — it is largely repeatable. (2023-24 xFG is partly in-sample, about 12 % of those shots were training data, which slightly flatters that season.)

What the model does not know: defender distance, catch-and-shoot vs off-the-dribble beyond the `ACTION_TYPE` label, fatigue, and play context. Player-tracking data would close much of that gap, but it is not public.

🔍 Critic review — Assess

Paste this phase (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Assess** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Assess step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether the champion was selected on an appropriate criterion, whether the test partition and out-of-time season were used only once, whether calibration was checked, and whether the player-level findings are interpreted with appropriate caveats. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Calibration only checked on average; the leaderboard could be noise; explain the modest AUC gain.	Added decile calibration on 2024-25 (all gaps ≤ 0.9 pts); added season-to-season persistence check (r = 0.68); explained irreducible randomness and log-loss gain.
2	`<fresh-chat critique>`	`<changes>`
3		

Beyond SEMMA — Deployment

SEMMA stops at Assess (scoring is part of it); the assignment requires delivery all the way to deployment. We publish the champion as a small web app: pick a spot on the court and a shot type → expected FG% and expected points, plus the 2024-25 shot-making leaderboard.

In [32]:
```
import joblib
champion = fitted[CHAMPION]
joblib.dump(champion, APP / "model.joblib", compress=3)
actions = train["ACTION_TYPE"].value_counts().index.astype(str).tolist()
meta = {"champion": CHAMPION, "created": datetime.now().isoformat(timespec="seco
        "validation": comparison.loc[CHAMPION].to_dict(), "final": final_table.t
        "actions": actions, "trained_on": f"{len(train):,} shots, seasons {POPUL
        "versions": {"python": platform.python_version(), "numpy": np.__version_
(APP / "model_metadata.json").write_text(json.dumps(meta, indent=2, default=str)
```

```
players.reset_index().to_csv(APP / "leaderboard.csv", index=False)
print(f"model.joblib: {(APP / 'model.joblib').stat().st_size / 1e6:.1f} MB")
```

```
model.joblib: 0.3 MB
```

In [33]:

```
APP_CODE = r"""
import json
from pathlib import Path
import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import streamlit as st
from matplotlib.patches import Arc, Circle, Rectangle
from shot_features import engineer   # score code - must sit next to this file

HERE = Path(__file__).parent
st.set_page_config(page_title="NBA xFG%", page_icon="🏀")

@st.cache_resource
def load():
    return joblib.load(HERE / "model.joblib"), json.loads((HERE / "model_metadat

model, meta = load()
st.title("🏀 NBA expected field-goal %")
st.caption(f"{meta['champion']} · trained on {meta['trained_on']}")

def court(ax):
    for p in [Circle((0, 0), 7.5, fill=False), Rectangle((-80, -47.5), 160, 190,
              Arc((0, 142.5), 120, 120, theta1=0, theta2=180), Arc((0, 0), 80, 8
              Rectangle((-220, -47.5), 0, 140), Rectangle((220, -47.5), 0, 140),
              Rectangle((-250, -47.5), 500, 470, fill=False)]:
        p.set_edgecolor("#555"); ax.add_patch(p)
    ax.set_xlim(-250, 250); ax.set_ylim(-47.5, 422.5); ax.set_aspect("equal"); a

shot, board = st.tabs(["Evaluate a shot", "2024-25 shot-making leaderboard"])
with shot:
    left, right = st.columns([1, 1])
    with left:
        x_ft = st.slider("Feet left (-) / right (+) of the hoop", -25.0, 25.0, 0
        y_ft = st.slider("Feet out from the hoop", -4.0, 40.0, 15.0, 0.5)
        action = st.selectbox("Shot type", meta["actions"], index=0)
        period = st.selectbox("Period", [1, 2, 3, 4, 5], index=0, format_func=la
        minutes = st.number_input("Minutes left in period", 0, 12 if period <= 4
        seconds = st.number_input("Seconds", 0, 59, 0)
    raw = pd.DataFrame([{"LOC_X": x_ft * 10, "LOC_Y": y_ft * 10, "ACTION_TYPE":
                         "MINUTES_REMAINING": minutes, "SECONDS_REMAINING": seco
    feats = engineer(raw).iloc[0]
    xfg = float(model.predict_proba(raw)[0, 1])
    value = 3 if feats["is_three"] else 2
    with right:
        fig, ax = plt.subplots(figsize=(4, 4))
        court(ax); ax.scatter([x_ft * 10], [y_ft * 10], s=120, c="#E4572E", zord
        st.pyplot(fig)
    c1, c2, c3 = st.columns(3)
    c1.metric("Expected FG%", f"{xfg:.1%}")
    c2.metric("Shot value", f"{value} pts" + (" (corner)" if feats["is_corner_th
    c3.metric("Expected points", f"{xfg * value:.2f}")
    st.caption(f"Distance {feats['dist_ft']:.1f} ft · angle {feats['angle_deg']:
with board:
```

```
    lb = pd.read_csv(HERE / "leaderboard.csv")
    st.write("Players with 500+ attempts. **Points over expected** = actual poin
             "an average shooter would score on the same shots.")
    st.dataframe(lb[["PLAYER_NAME", "team", "FGA", "FG_pct", "xFG_pct", "FG_over
                 hide_index=True)
"""
(APP / "app.py").write_text(APP_CODE.strip() + "\n")
v = meta["versions"]
(APP / "requirements.txt").write_text(f"streamlit\nscikit-learn=={v['scikit-lear
compile(APP_CODE, "app.py", "exec")
print("App files:", sorted(p.name for p in APP.iterdir() if p.is_file()))
print(f"When deploying, choose Python {v['python'][:4]} so the pinned versions i
```

```
App files: ['app.py', 'leaderboard.csv', 'model.joblib', 'model_metadata.json',
'requirements.txt', 'shot_features.py']
When deploying, choose Python 3.13 so the pinned versions install and the model l
oads.
```

In [34]:
```
# Smoke test: reload exactly as the app does and score three hand-made shots
reloaded = joblib.load(APP / "model.joblib")
demo = pd.DataFrame([{"LOC_X": 0, "LOC_Y": 5, "ACTION_TYPE": "Driving Layup Shot
                      {"LOC_X": -225, "LOC_Y": 20, "ACTION_TYPE": "Jump Shot", "P
                      {"LOC_X": 30, "LOC_Y": 400, "ACTION_TYPE": "Jump Shot", "PE
                     index=["layup at the rim", "left-corner three", "half-court
demo.assign(xFG=reloaded.predict_proba(demo)[:, 1].round(3))
```

Out[34]:

	LOC_X	LOC_Y	ACTION_TYPE	PERIOD	MINUTES_REMAINING	SECONDS_REMAI
layup at the rim	0	5	Driving Layup Shot	1	8	
left-corner three	-225	20	Jump Shot	2	5	
half-court heave	30	400	Jump Shot	2	0	

Deploy (free): put `artifacts/app/` in your GitHub repo → share.streamlit.io → *Create app* → choose `.../app/app.py`; under *Advanced settings* pick the Python version printed above. `shot_features.py` must stay next to `app.py` (it is the score code the model was trained with).

Monitoring idea: each season, compare league-wide mean xFG% with actual FG% (calibration-in-the-large) and re-train if they drift apart by more than ~1 percentage point (e.g. rule changes, new shot trends).

Appendix — SAS Enterprise Miner demo

*SAS Enterprise Miner is available through **SAS OnDemand for Academics** when your professor registers the class (SAS Studio alone is free to individuals but has no Enterprise*

Miner). The cell below exports the exact modeling sample, so the SAS flow uses the same data.

In [35]:
```
sas = sample.drop(columns=["GRID_TYPE", "SHOT_ATTEMPTED_FLAG"]).join(engineer(sa
sas.to_csv(DATA / "nba_shots_sample_for_sas.csv", index=False)
print(f"Exported {len(sas):,} rows x {sas.shape[1]} columns -> {DATA / 'nba_shot
      f"({(DATA / 'nba_shots_sample_for_sas.csv').stat().st_size / 1e6:.0f} MB)"
```

```
Exported 199,999 rows x 34 columns -> artifacts/data/nba_shots_sample_for_sas.csv
(57 MB)
```

Rebuild the flow in Enterprise Miner:

1. **New project** → create a library → **File Import** node with `nba_shots_sample_for_sas.csv`.
2. **Input Data / variable roles:** `SHOT_MADE_FLAG` = Target (binary); `EVENT_TYPE`, IDs, names, dates, `SEASON` = Rejected (same reasons as M.1); engineered columns = Input.
3. **Sample:** Data Partition node — 60/20/20 (EM partitions rows randomly; note the difference from the game-grouped split here).
4. **Explore:** StatExplore, MultiPlot, Variable Selection, Cluster (on `LOC_X`, `LOC_Y`).
5. **Modify:** Replacement (rare `action` levels), Transform Variables (e.g. log of `dist_ft`), Impute.
6. **Model:** Regression, Decision Tree, Neural Network, Gradient Boosting → **Ensemble**.
7. **Assess:** Model Comparison (selection statistic: validation misclassification or average squared error ≈ Brier), then **Score** on the 2024-25 season.
8. **Report:** screenshot the diagram and the Model Comparison results; compare them with section A.1 in your article.

Submission checklist

- [x] Analysis written for every step (author/course/URLs still to add in the header)
- [x] Critic round 1 done for all five steps (`critiques/round_1.md`) — **rounds 2–3 to run in a fresh Claude chat**
- [x] *Run all* completes without errors (≈ 80 s here; ≈ 3–5 min on free Colab)
- [x] `artifacts/` committed (figures, reports, model, app) — `artifacts/data/` ignored
- [ ] App deployed; URL in the header and README
- [ ] SAS Enterprise Miner demo (if the class is registered) with screenshots
- [ ] Medium article published + YouTube walkthrough linked